ARP Éducation

Documentation fonctionnelle et technique

Application web PWA — Next.js + SQLite — gestion de scolarité

Projet — cahier des charges ARP éducation

Version 2.0 — 19 octobre 2026

Sommaire

1. Présentation du projet
2. Besoins fonctionnels
3. Besoins non fonctionnels
4. Matrice des autorisations
5. Modèle de données — les 13 entités
6. Règles de gestion et cycles de vie
7. Architecture technique
8. Interface HTTP (pages et API)
9. Sécurité
10. PWA : installation et mode hors ligne
11. Installation, démarrage, comptes de démonstration
12. Tests réalisés et résultats
13. Limites connues et évolutions possibles
14. Annexe — arborescence du projet

1. Présentation du projet

ARP Éducation est une application web de gestion de scolarité couvrant six domaines demandés par le cahier des charges : utilisateurs, présences et emplois du temps, matières/salles/notes, comptes rendus (dépôts), réclamations et notifications. Elle est développée **en un seul projet Next.js** assurant à la fois le front et le back, s'appuie sur une **base de données intégrée SQLite** et se présente comme une **application installable (PWA)** fonctionnant **en ligne et hors ligne**, sur PC comme sur téléphone.

1.1 Choix structurants

Exigence du cahier des charges	Décision réelle	Motivation
Base de données intégrée **H2**	Substituée par **SQLite** (`data/arp.db`)	H2 n'existe qu'en Java : il n'existe aucun binding Node.js exploitable. SQLite est l'équivalent fonctionnel en contexte Node — base embarquée dans un fichier local, sans serveur à installer. Choix validé par le commanditaire.
Next.js récent	**Next 13.5.11** (App Router)	La machine cible reste en **Node 18.13** (refus de monter de version). Les versions actuelles de Next exigent Node 18.17+. Next 13.5 est la dernière branche compatible. Conséquence : pas d'actions serveur, remplacées par des **route handlers** (choix assumé et homogène).
Installable comme YouTube, PC et téléphone	**PWA** manifest + service worker écrit à la main	Répond à l'exigence sans framework PWA externe.
Fonctionner avec ou sans connexion	Cache des pages consultées + page hors ligne affichant le **calendrier de la semaine**	Le hors ligne est en **consultation**; les écritures requièrent le réseau.

1.2 Synthèse technologique

Couche	Technologie	Version installée
Framework full-stack	Next.js (App Router, React Server Components)	13.5.11
Vue / runtime	React et React DOM	18.3.1
Langage	TypeScript (strict)	5.9.3
Base de données	SQLite via better-sqlite3 (mode WAL, clés étrangères activées)	9.6.0
Style	Tailwind CSS (utilitaires + classes de composants maison)	3.4.19
Authentification	Implementation maison : cookie HMAC-SHA256 + hachage scrypt	—

Exposeur HTTPS	Serveur Node personnalisé `server-https.js` + certificat mkcert	Node 18.13

2. Besoins fonctionnels

Le tableau reprend besoin par besoin l'expression du cahier des charges, l'acteur concerné et l'état de réalisation dans l'application.

ID	Besoin	Acteur	Priorité	Réalisation
BF-01	Un compte administrateur unique, fixe, pré-existant	Système	Obligatoire	Compte `admin@arp.edu` présent au démarrage ; impossible à créer ou supprimer par l'interface
BF-02	Créer les comptes professeur et étudiant à partir de leur adresse électronique	Admin	Obligatoire	Module Utilisateurs ; le mot de passe n'est **pas** saisi : le champ reste vide
BF-03	Envoyer un courriel demandant la définition du mot de passe	Système	Obligatoire	Invitation enregistrée et consultable (module Emails) avec lien `/reset?token=…`
BF-04	Le destinataire définit son mot de passe, puis se connecte	Prof / Étudiant	Obligatoire	Page publique `/reset` ; jeton à usage unique ; connexion bloquée tant que le mot de passe n'est pas défini
BF-05	Renvoyer l'invitation si nécessaire	Admin	Souhaité	Bouton « Renvoyer l'email » sur la liste des utilisateurs
BF-06	Définir les salles	Admin	Obligatoire	Module Salles : nom, bâtiment, capacité, type (salle, amphi, laboratoire)
BF-07	Créer chaque année des classes à partir des comptes étudiants	Admin	Obligatoire	Module Classes : nom, filière, année scolaire
BF-08	Sélectionner les étudiants dans une classe, à tout moment	Admin	Obligatoire	Interface à cases à cocher ; la transaction retire l'étudiant de son ancienne classe avant de l'affecter
BF-09	Créer les matières avec coefficient, professeur, salles et volume horaire	Admin	Obligatoire	Module Matières : code, nom, coefficient, heures, classe, professeur responsable
BF-10	Notifier par courriel après création d'une matière	Système	Obligatoire	Courriel d'affectation envoyé au professeur, avec lien vers son module Notes
BF-11	Chaque semaine, définir le programme de la classe	Admin	Obligatoire	Éditeur hebdomadaire : choix classe + lundi de semaine, deux créneaux par jour
BF-12	Maximum deux matières par jour, trois heures chacune	Règle	Obligatoire	Créneaux imposés 09:00–12:00 et 13:00–16:00 ; contrainte d'unicité (classe, semaine, jour, créneau)
BF-13	Pouvoir récupérer l'emploi du temps en PDF	Tous	Obligatoire	Bouton « Télécharger en PDF » : le serveur génère un fichier `.pdf` A4 paysage (route `/api/emplois/pdf`) que le navigateur enregistre directement ; un bouton « Imprimer » reste disponible
BF-14	Le professeur consulte ses affectations	Prof	Obligatoire	Matières, Notes, Présences et Comptes rendus filtrés sur les matières dont il est responsable
BF-15	Signaler les absences dans l'application	Prof	Obligatoire	Appel par séance (présent / absent / retard) sur la date choisie, enregistrement par upsert
BF-16	Créer un compte rendu avec une durée de validité	Prof	Obligatoire	Module Comptes rendus : titre, consigne, matière, date limite de dépôt
BF-17	Après expiration, vérifier les dépôts et attribuer une note	Prof	Obligatoire	Les dépôts deviennent lisibles et notables (0–20 + commentaire) uniquement après la date limite
BF-18	À la fin du cours, soumettre la clôture avec les notes d'examen final	Prof	Obligatoire	Formulaire de clôture listant tous les étudiants de la classe ; passe la matière au statut « terminée »
BF-19	Consulter ses absences	Étudiant	Obligatoire	Module Présences : compteurs et historique daté avec professeur
BF-20	Consulter ses notes par matière et le total	Étudiant	Obligatoire	Module Notes : détail CC/TP/Examen, moyenne par matière, moyenne générale pondérée par coefficient
BF-21	Déposer son compte rendu	Étudiant	Obligatoire	Dépôt possible jusqu'à la date limite, une fois par étudiant et par compte rendu

BF-22	Télécharger son emploi du temps	Étudiant	Obligatoire	Vue filtrée de force sur sa classe (une requête visant une autre classe est ignorée), téléchargement du PDF, mémorisation hors ligne de la page
BF-23	L'administration valide la matière clôturée avant publication	Admin	Obligatoire	Bouton « Valider et publier aux étudiants » ; les notes restent invisibles tant que le statut n'est pas « validée »
BF-24	Seules les notifications créées par l'admin sont émises ; elles sont vues par tous	Admin	Obligatoire	Formulaire réservé à l'admin ; liste sans filtre de destination
BF-25	Seuls les professeurs et étudiants créent des réclamations ; visibles par tous	Prof / Étudiant	Obligatoire	Formulaire masqué et refusé côté serveur pour l'admin ; liste complète pour tous les rôles
BF-26	Terminer la semaine précédente comme base de travail	Admin	Complément	Bouton « Copier la semaine précédente » (ajout pratique)
BF-27	Afficher un tableau de bord adapté au rôle	Tous	Complément	Indicateurs, calendrier de la semaine et dernières notifications dès la page d'accueil

2.1 Besoins implicites ajoutés (non écrits dans le cahier des charges)

- **Désactivation d'un compte** plutôt que suppression, pour conserver l'historique de notes et de présences.
- **Suppression protégée** des matières et salles : l'opération est refusée si des lignes dépendantes existent, avec message d'erreur explicite renvoyé vers la page.
- **Validation serveur systématique** : bornes de notes 0–20, types de pièce et de salle contraints, dates obligatoires.
- **Redoublement de contrôle côté écriture** : chaque route rejette un rôle non autorisé par un code HTTP 403, indépendamment de ce que masque l'interface.
- **Réinitialisation de la démonstration** : supprimer `data/arp.db` recrée jeu de démonstration complet.

3. Besoins non fonctionnels

ID	Exigence	Implémentation
BNF-01	Application installable (PC et mobile)	Manifest `/manifest.webmanifest` : nom, icône 1024 px en versions « any » et « maskable », affichage **standalone**, thème `#4f46e5`. Enregistrement du service worker par un micro-composant client.
BNF-02	Utilisable hors connexion	Service worker maison : navigations en « réseau d'abord, puis cache, puis page hors ligne » ; ressources statiques en « cache d'abord ». Cache versionné `arp-v2`.
BNF-03	Le hors ligne doit afficher le calendrier de la semaine	La page `/offline` rend la grille 5 jours × 2 créneaux, vidée ou remplie selon la dernière session en ligne.
BNF-04	Adapté au téléphone	Mise en page responsive : barre latérale sombre ≥ 768 px, barre de navigation basse défilante en dessous ; zones tactitaires confortables ; tableaux à défilement horizontal.
BNF-05	Sécurité de session	Cookie **httpOnly**, **SameSite=Lax**, signature HMAC-SHA256, durée 30 jours ; mots de passe hachés avec scrypt et sel aléatoire ; comparaison à temps constant.
BNF-06	Aucune exposition de requête après rechargement	Tout écriture répond par une redirection **303** ; le motif est précisé en section 7.3.
BNF-07	Zéro serveur de base à administrer	SQLite embarqué en mode WAL ; schéma auto-créé ; jeu de données auto-inséré au premier lancement.
BNF-08	Transparent sur le canal d'envoi réel	Courriels simulés et historisés dans une table dédiée, consultable par l'admin ; le document explique le passage à SMTP.
BNF-09	Langue	Interface intégralement en français, dates au format français.

4. Matrice des autorisations

Légende : **W** = écriture autorisée, **R** = lecture, **—** = refus (l'API répond 403).

Fonctionnalité	Admin	Professeur	Étudiant
Créer / modifier / supprimer des comptes	W	—	—
Définir les salles	W	R	R
Créer les classes, affecter les étudiants	W	R	—
Créer les matières, valider une matière clôturée	W	R	R
Planifier l'emploi du temps hebdomadaire	W	R	R
Télécharger le PDF d'une classe	R (toute classe)	R (toute classe)	R (sa classe uniquement)
Clôturer une matière (notes d'examen)	— (sauf matière sans professeur)	W (ses matières)	—
Écrire dans les notes d'une matière validée	—	—	— (notes gelées)
Faire l'appel (présences)	W	W (ses matières)	R (les siennes)
Saisir les notes CC / TP	W	W (ses matières)	R (après validation)
Clôturer une matière (notes d'examen)	—	W (ses matières)	—
Créer un compte rendu, le corriger	W	W (ses matières)	—
Déposer un compte rendu	—	—	W (avant la limite)
Créer une réclamation	—	W	W
Traiter une réclamation (statut) / supprimer	W	R	R
Publier une notification	W	—	—
Consulter les notifications	R	R	R
Consulter les courriels envoyés	W	—	—

5. Modèle de données

Treize tables. Les relations sont déclarées en clés étrangères avec intégrité référentielle active (`PRAGMA foreign_keys = ON`). Le typage SQLite est `INTEGER` / `TEXT` / `REAL` ; les dates sont stockées en texte ISO.

5.1 Vue d'ensemble des relations

Entité	Rôle	Relations
classes	Groupe d'étudiants d'une filière pour une année	1 — N users, 1 — N matieres, 1 — N emplois
users	Compte (admin, professeur ou étudiant)	N — 1 classes (étudiants) ; 1 — N matieres (professeurs responsables) ; 1 — N notes, presences, reclamations, cr_depots
reset_tokens	Jeton d'invitation / changement de mot de passe	N — 1 users
emails	Boîte d'envoi (courriels simulés)	destinataire par adresse électronique, sans clé étrangère
matieres	Enseignement avec coefficient, volume horaire et statut	N — 1 classes, N — 1 users (professeur), 1 — N salles via emplois, 1 — N notes, presences, cr_assignments
salles	Lieu physique	1 — N emplois
emplois	Créneau d'une semaine donnée	N — 1 classes, N — 1 matieres, N — 1 salles
presences	Pointage d'un étudiant à une séance	N — 1 matieres, N — 1 users (étudiant), N — 1 users (professeur relevé)
notes	Note CC, TP ou examen	N — 1 matieres, N — 1 users
cr_assignments	Compte rendu demandé avec date limite	N — 1 matieres, N — 1 users (professeur), 1 — N cr_depots
cr_depots	Remise d'un étudiant, notée après expiration	N — 1 cr_assignments, N — 1 users
reclamations	Réclamation ouverte par un professeur ou étudiant	N — 1 users
notifications	Annonce globale créée par l'administration	sans destinataire : visible par tous

5.2 Dictionnaire détaillé

classes

Champ	Type	Contraintes	Description
id	INTEGER	PK autoincrément	Identifiant
nom	TEXT	NOT NULL, UNIQUE	Sigle du groupe, ex. INFO-2A
filiere	TEXT	NOT NULL	Filière, ex. Informatique
annee	TEXT	NOT NULL	Année scolaire, ex. 2026-2027

users

Champ	Type	Contraintes	Description
id	INTEGER	PK	Identifiant
nom / prenom	TEXT	NOT NULL	Identité affichée
email	TEXT	NOT NULL, UNIQUE	Identifiant de connexion, normalisé en minuscules
mot_de_passe	TEXT	**NULL autoriser**	`sel:hachage` scrypt ; NULL tant que l'invitation n'est pas activée
role	TEXT	CHECK in (admin, prof, etudiant)	Rôle applicatif
classe_id	INTEGER	FK classes	Classe de l'étudiant ; NULL pour un professeur ou l'admin
actif	INTEGER	NOT NULL défaut 1	Compte désactivé = refus de connexion

reset_tokens

Champ	Type	Contraintes	Description
token	TEXT	PK	Hasard cryptographique, 48 caractères hexadécimaux
user_id	INTEGER	FK users, NOT NULL	Compte concerné
cree_le	TEXT	NOT NULL	Date d'émission
utilise	INTEGER	NOT NULL défaut 0	Passe à 1 après usage ; un jeton utilisé est refusé

emails

Champ	Type	Description
destinataire	TEXT	Adresse électronique visée
sujet / contenu	TEXT	Corps du message généré
lien	TEXT	Chemin cliquable (activation, module cible)
cree_le	TEXT	Horodatage d'envoi

matieres

Champ	Type	Contraintes	Description
code	TEXT	NOT NULL, UNIQUE	Sigle, forcé en majuscules (DEVWEB)
nom	TEXT	NOT NULL	Libellé
coefficient	INTEGER	défaut 1, interface 1–6	Pondération de la moyenne générale
heures	INTEGER	défaut 6, interface ≥ 1	Volume horaire prévu
statut	TEXT	CHECK in (en_cours, termine, valide)	Cycle de vie — voir section 6
prof_id	INTEGER	FK users	Responsable pédagogique
classe_id	INTEGER	FK classes, NOT NULL	Groupe concerné

salles

Champ	Type	Contraintes	Description
nom	TEXT	NOT NULL, UNIQUE	Étiquette de salle
batiment	TEXT	NOT NULL	Lettre ou code du bâtiment
capacite	INTEGER	NOT NULL	Nombre de places
type	TEXT	CHECK in (salle, amphi, labo)	Nature du local

emplois

Champ	Type	Contraintes	Description
semaine_debut	TEXT	NOT NULL	Date du lundi de la semaine ISO
jour	TEXT	CHECK in (LUNDI…VENDREDI)	Jour de semaine
creneau	INTEGER	CHECK in (1,2)	1 = matin, 2 = après-midi
heure_debut / heure_fin	TEXT	NOT NULL	Imposés par le code : 09:00–12:00 ou 13:00–16:00
classe_id, matiere_id, salle_id	INTEGER	FK, NOT NULL	Objet du créneau
—	—	UNIQUE (classe_id, semaine_debut, jour, creneau)	Deux matières au plus par jour : la base l'interdit, l'interface non plus n'offre que deux lignes

presences

Champ	Type	Contraintes	Description
date	TEXT	NOT NULL	Date de séance

statut	TEXT	CHECK in (present, absent, retard)	Pointage
matiere_id / etudiant_id	INTEGER	FK, NOT NULL	Séance et étudiant
prof_id	INTEGER	FK users	Enseignant ayant relevé l'appel
—	—	UNIQUE (date, matiere_id, etudiant_id)	Un seul pointage par étudiant, séance et date ; l'enregistrement actualise

notes

Champ	Type	Contraintes	Description
valeur	REAL	CHECK BETWEEN 0 AND 20	Note sur 20, pas de 0,25 acceptés
type	TEXT	CHECK in (CC, TP, EXAM)	Contrôle continu, travaux pratiques, examen final
date	TEXT	NOT NULL	Date d'évaluation
matiere_id / etudiant_id	INTEGER	FK, NOT NULL	Évaluation

cr_assignments

Champ	Type	Description
titre / consigne	TEXT NOT NULL	Intitulé et sujet à rendre
matiere_id / prof_id	INTEGER FK	Matière et enseignant émetteur
date_limite	TEXT NOT NULL	Fin de validité du dépôt
cree_le	TEXT NOT NULL	Date de création

cr_depots

Champ	Type	Contraintes	Description
assignment_id / etudiant_id	INTEGER	FK, NOT NULL	Remise
contenu	TEXT	NOT NULL	Texte déposé par l'étudiant
rendu_le	TEXT	NOT NULL	Horodatage du dépôt
note	REAL	CHECK BETWEEN 0 AND 20	NULL tant que non corrigé
feedback	TEXT	NULL autorisé	Retour du correcteur
—	—	UNIQUE (assignment_id, etudiant_id)	Un dépôt par compte rendu et par étudiant

reclamations

Champ	Type	Contraintes	Description
objet / message	TEXT	NOT NULL	Réclamation
type	TEXT	CHECK in (note, emploi, salle, autre)	Nature du sujet
statut	TEXT	défaut ouvert, CHECK in (ouvert, en_cours, traite)	Suivi assuré par l'admin
user_id	INTEGER	FK users, NOT NULL	Auteur (professeur ou étudiant)
cree_le	TEXT	NOT NULL	Date de dépôt

notifications

Champ	Type	Contraintes	Description
titre / message	TEXT	NOT NULL	Annonce
lue	INTEGER	défaut 0	Champ réservé (les annonces étant globales, l'accusé individuel n'est pas exposé)
cree_le	TEXT	NOT NULL	Horodatage de publication

5.3 Jeu de démonstration initial

Au premier démarrage, `lib/db.ts` insère 2 classes, 1 admin, 3 professeurs, 5 étudiants, 4 matières, 3 salles, l'emploi du temps des semaines courante et précédente, 3 pointages, 6 notes, 2 comptes rendus dont 2 dépôts (un corrigé), 2 réclamations, 2 notifications et 1 courriel archivé.

6. Règles de gestion et cycles de vie

6.1 Activation d'un compte

1. L'admin crée le compte (rôle professeur ou étudiant, classe pour un étudiant). Le mot de passe reste `NULL`.
2. La création génère un jeton, l'insère dans `reset_tokens` et consigne le courriel d'invitation contenant `/reset?token=…`.
3. Le destinataire ouvre le lien, saisit deux fois un mot de passe d'au moins 6 caractères.
4. Le mot de passe est haché, le jeton marqué consommé ; toute réutilisation du lien est refusée.
5. La connexion devient possible. Tant que le mot de passe n'est pas défini, la connexion est refusée avec un message évoquant le compte non activé.

6.2 Cycle de vie d'une matière

Statut	Déclencheur	Effet
en_cours	Création par l'admin	Saisie CC/TP autorisée au responsable ; appel possible ; création de comptes rendus possible ; les notes sont invisibles aux étudiants
termine	Clôture par le professeur, avec les notes d'examen des étudiants de la classe	En transaction : suppression des anciens EXAM de la matière, insertion des nouvelles notes, passage du statut. La matière apparaît dans la file « à valider » de l'admin
valide	Bouton « Valider et publier aux étudiants » de l'admin	Uniquement depuis `termine`. Les notes deviennent lisibles par les étudiants ; un courriel est consigné pour tracer la publication

Le retour en arrière n'est pas offert dans l'interface : le flux est volontairement à sens unique.

6.3 Cycle de vie d'un compte rendu

1. Le professeur (ou l'admin) crée le sujet avec `date_limite` ; le titre, la consigne et la matière sont obligatoires.
2. Avant la limite : chaque étudiant de la classe peut déposer, une seule fois. Le dépôt est refusé si le délai est dépassé ou si l'étudiant a déjà déposé.
3. Après la limite : les dépôts deviennent consultables par le responsable, qui attribue une note et un commentaire.
4. L'étudiant découvre note et retour dans son module Comptes rendus.

6.4 Droits de manipulation vérifiés côté serveur

- **Pages réservées** : `/users`, `/classes` et `/emails` exigent le rôle admin ; un professeur ou un étudiant qui les appelle directement est renvoyé vers le tableau de bord avec le motif « accès refusé ».
- **Emploi du temps** : écriture admin uniquement, et chaque créneau soumis est re-vérifié (matière appartenant bien à la classe, salle existante) — une soumission fabriquée à la main est refusée.
- **Notes** : un professeur ne peut saisir que dans ses matières, pour un étudiant inscrit dans la classe de la matière, et tant que la matière est « en cours ». La clôture appartient au professeur ; l'administrateur ne fait que valider. Après validation, plus aucune écriture sur les notes de la matière.
- **Présences** : appel limité aux matières dont le professeur a la charge ; un professeur ne supprime que les lignes de ses matières.
- **Comptes rendus** : création réservée au professeur responsable d'une matière en cours ; dépôt étudiant limité à sa classe, avant la date limite, une seule fois ; correction réservée au professeur responsable ou à l'admin.
- **Notifications** : création et suppression admin uniquement. **Réclamations** : création par professeur ou étudiant seulement, traitement (changement d'état, suppression) réservé à l'admin.

6.5 Règles de l'emploi du temps

- Deux créneaux par jour, cinq jours : la semaine compte donc dix lignes au maximum par classe.
- Les horaires ne sont pas saisis : ils dépendent du créneau (09:00–12:00, 13:00–16:00).

- Enregistrer une semaine **remplace** les créneaux de cette semaine pour la classe : la suppression puis l'insertion se font dans une transaction, ce qui explique aussi qu'un créneau laissé vide devienne « libre ».
- Une salle est obligatoire dès qu'une matière est placée.
- La semaine précédente peut être copiée puis ajustée.

6.6 Calcul des moyennes

Moyenne de matière = moyenne arithmétique des notes publiées de la matière (CC, TP et examen). Moyenne générale = moyenne pondérée des moyennes de matière par leur coefficient, sur les seules matières **validées**. Le total est affiché avec sa couleur de conformité (acquis si supérieur ou égal à 10).

7. Architecture technique

7.1 Organisation du code

Élément	Rôle
`app/…/page.tsx` (15)	Composants serveur : lecture directe de SQLite et rendu HTML. Aucun appel réseau client.
`app/api/…/route.ts` (13)	Points d'entrée d'écriture : analyse du formulaire, contrôle du rôle, requête préparée, redirection.
`lib/db.ts`	Ouverture unique et mémoïsée de la connexion, création du schéma, insertion du jeu de démonstration, hachage et vérification des mots de passe.
`lib/auth.ts`	Session (cookie signé), connexion, création de jeton d'invitation, définition du mot de passe par jeton, helper de date ISO du jour et du lundi de semaine.
`lib/mail.ts`	Consignation d'un courriel dans la table `emails` (simulation de l'envoi).
`lib/origin.ts`	Restitution de l'origine réelle de la requête pour que les redirections conviennent aussi au téléphone.
`middleware.ts`	Garde-frontière : page ou API non publique sans session valide = redirection vers `/login`.
`components/`	Sidebar (navigation desktop et basse mobile), PwaRegister (enregistrement du service worker), PrintButton (fenêtre d'impression).
`lib/pdf.ts`	Générateur PDF maison (PDF 1.4, polices de base Helvetica et Helvetica-Bold, encodage WinAnsi pour les accents, calcul des largeurs de glyphes) : aucune dépendance externe ajoutée au projet.
`public/`	Manifest, service worker, icône.
`server-https.js`	Démarrage HTTPS de la version produite sur le port 3443 avec le certificat mkcert.

7.2 Flux d'une écriture

`<form method="post" action="/api/x">` → le middleware vérifie la session → la route handler lit `req.formData()`, contrôle le rôle et la responsabilité pédagogique, exécute une instruction préparée (paramètres liés, jamais de concaténation), `revalidatePath()` sur les pages concernées, puis répond `303 See Other`. Résultat : rafraîchir la page ne rejoue pas l'écriture, et le visiteur retombe sur une page HTML.

7.3 Pourquoi 303 plutôt qu'un autre code

> Next 13 applique `NextResponse.redirect()` en **307** par défaut, ce qui invite le navigateur à **rejouer le POST** sur le chemin de la page. La page étant un composant serveur, le corps du formulaire n'est plus recevable : on obtenait une erreur 500 après chaque connexion et chaque formulaire. **Tous** les points d'écriture passent donc explicitement en 303, qui force un rechargement en GET. Cette précision a été la source du plus grand nombre de défauts rencontrés.

7.4 Rendu et révalidation

Chaque page portant la directive `force-dynamic` est rendue à la demande : les données affichées ne peuvent pas être périmées. Après une écriture, `revalidatePath` purge les cheminées concernées par la même requête (écrite une matière → pages matières et notes purgées).

8. Interface HTTP

8.1 Pages

Chemin	Contenu	Accès
`/`	Tableau de bord selon le rôle : indicateurs, calendrier de la semaine, dernières notifications	connecté
`/login`	Connexion, bandeaux d'erreur et d'activation réussie, rappel des comptes de démonstration	public
`/reset`	Définition du mot de passe depuis un jeton	public
`/offline`	Message hors ligne et grille horaire de la semaine	public
`/users`	Liste, création, invitation, renvoi, activation/désactivation, suppression	admin
`/classes`	Classes par onglet, affectation par cases à cocher, effectif et état d'activation	admin
`/emails`	Boîte d'envoi avec lien activable	admin
`/matieres`	Liste filtrée par rôle, file de validation, création avec coefficient, heures, classe, professeur	tous
`/salles`	Inventaire, usage par créneaux, création/suppression	écritures admin
`/emplois`	Grille hebdomadaire, sélection classe et semaine, éditeur admin, téléchargement du PDF	tous
`/presences`	Appel par matière et date pour l'enseignant ; historique et compteurs pour l'étudiant	tous
`/notes`	Saisie CC/TP, tableau de synthèse, clôture avec notes d'examen ; moyenne publiée pour l'étudiant	tous
`/comptes-rendus`	Création avec date limite, dépôts, correction, restitution à l'étudiant	tous
`/reclamations`	Création par professeur/étudiant, suivi de statut par l'admin, liste visible de tous	tous
`/notifications`	Publication par l'admin, consultation globale	tous

8.2 Opérations d'écriture

Point d'entrée	Actions	Rôle autorisé
`/api/auth/login`	ouverture de session	public
`/api/auth/logout`	fermeture	connecté
`/api/auth/reset`	définition du mot de passe par jeton	public
`/api/users`	create, renvoyer, toggle, delete	admin
`/api/classes`	create, affecter, delete	admin
`/api/matieres`	create, valider, delete	admin
`/api/salles`	create, delete	admin
`/api/emplois`	save-week, copy-prev	admin
`/api/emplois/pdf` (GET)	génère le PDF de la semaine et le renvoie en pièce jointe	tous, l'étudiant étant forcé à sa classe
`/api/presences`	save, delete	admin ; professeur pour ses matières
`/api/notes`	create (CC/TP), cloturer (EXAM), delete	admin ; professeur responsable pour saisie et clôture
`/api/comptes-rendus`	create, depot, noter, delete	professeur/admin pour create, noter, delete ; étudiant pour depot
`/api/reclamations`	create, statut, delete	professeur/étudiant pour create ; admin pour statut et delete
`/api/notifications`	create, delete	admin

9. Sécurité

Domaine	Mise en œuvre
Mots de passe	scrypt (32 octets) avec sel de 16 octets par compte ; vérification par comparaison à temps constant ; aucun mot de passe en clair stocké ni affiché.
Session	Cookie `arp_session` = `id.signature`, signature HMAC-SHA256 tronquée, `httpOnly`, `SameSite=Lax`, chemin racine, 30 jours. Impossible à forger côté client ; la signature est recontrôlée à chaque requête.
Invitations	Jeton aléatoire de 24 octets (hex), stocké, marqué consommé après usage, lien périmé ensuite.
Contrôle d'accès	Double niveau : le middleware bloque l'accès aux pages et API, chaque route handler exige le rôle attendu (et la responsabilité pédagogique pour notes, présences et comptes rendus) et répond 403 sinon. Masquage d'interface seul n'est jamais la garantie.
Injections	100 % des requêtes passent par des instructions préparées à paramètres liés ; aucune concaténation de valeur utilisateur dans du SQL.
Intégrité	Clés étrangères actives, contraintes CHECK sur tous les domaines de valeur, bornes 0–20 sur les notes, unicité contre les doublons de pointage, dépôt et créneau.
Transactions	Affectation d'un étudiant à une classe, enregistrement d'une semaine, clôture d'une matière : écritures atomiques.
XSS	React échappe les interpolations ; les retours à la ligne sont rendus par une classe CSS (`whitespace-pre-wrap`) et non par injection HTML.

Limites de sécurité à connaître

- La clé HMAC utilise une valeur par défaut de démonstration. En production il faut définir une variable d'environnement `ARP_SECRET` forte ; sinon une session pourrait être fabriquée par un tiers connaissant le schéma.
- Les comptes de démonstration partagent un mot de passe faible, volontairement documentés sur l'écran de connexion.
- Le dépôt `certs/` contient une clé privée ; il est exclu du suivi de versions et ne doit pas être synchronisé ni transmis.
- SQLite n'offre pas de contrôle d'accès par utilisateur : tout processus lisant `data/arp.db` peut lire les hachages.

10. PWA : installation et mode hors ligne

10.1 Comment l'installation est rendue possible

Les trois conditions du navigateur sont remplies : manifest valide avec icône et affichage `standalone`, service worker actif avec gestionnaire de requêtes, contexte sécurisé. Sur PC, l'origine `localhost` est considérée sécurisée, ce qui permet d'installer sans certificat. Sur Android, **localhost n'existe pas pour le téléphone** et une adresse locale en HTTP ne constitue pas un contexte sécurisé : le HTTP y rend l'installation et l'occupation hors ligne impossibles. C'est la raison de l'exposeur HTTPS local.

10.2 Stratégie de cache du service worker

Type de requête	Stratégie	Comportement constaté
Fichier essentiel (manifest, icône, `/offline`)	Pré-caché à l'installation	Toujours disponible
Navigation HTML	Réseau d'abord, puis cache, puis page hors ligne	Page déjà vue : servie hors ligne ; page jamais vue : message hors ligne avec calendrier de la semaine
Ressources statiques	Cache d'abord, puis réseau	Chargement instantané, pas de consommation de données

Le cache est versionné (`arp-v2`) : une nouvelle publication évince automatiquement les entrées anciennes.

10.3 Accès depuis un téléphone

1. Lancer l'exposeur HTTPS : `npm run build` puis `npm run start:https` ; l'application écoute sur le port 3443.
2. Traiter `certs/arp-root-ca.pem` comme le fichier à transférer sur le téléphone et l'installer en tant que **certificat d'autorité** (Paramètres → Sécurité → Chiffrement et informations d'identification → Installer un certificat).
3. Sur le téléphone, même réseau que le PC, ouvrir `https://192.168.100.10:3443` : aucun avertissement de sécurité n'apparaît.
4. Menu de Chrome → « Installer l'application ». Consulter une fois l'emploi du temps pour le préparer au hors ligne.

Le certificat est généré par mkcert (autorité locale créée sur le poste, certificat valable jusqu'au 19 janvier 2029 pour l'adresse 192.168.100.10, 127.0.0.1, localhost et le nom de la machine). Si l'adresse IP du PC change, il suffit de relancer `scripts\make-cert.ps1`, qui la détecte et régénère le certificat.

10.4 Ce qui marche hors ligne, ce qui ne marche pas

Marche : consultation de toute page déjà ouverte en ligne — calendrier de la semaine, notes, présences, matières, comptes rendus, notifications et réclamations. L'impression navigateur de l'emploi du temps reste possible hors ligne ; la génération du fichier PDF demande une connexion au serveur.

Ne marche pas : les écritures, qui interrogent le serveur (appel, dépôt, note, création). Le service worker ne met jamais en file d'attente des modifications à rejouer : le choix est délibéré, car un pointage non confirmé silencieusement serait trompeur pour l'enseignant.

11. Installation, démarrage, démonstration

Besoin	Commande	Url obtenue
Développement (rechargement à chaud)	`npm install` puis `npm run dev`	http://localhost:3000
Production locale PC	`npm run build` puis `npm run start`	http://localhost:3000
Production avec téléphone (HTTPS)	`npm run build` puis `npm run start:https`	https://localhost:3443 et https://192.168.100.10:3443
Réinitialiser les données de démonstration	Arrêter le serveur et supprimer `data/arp.db`	le schéma et le jeu sont recréés au lancement suivant

Comptes de démonstration (mot de passe `123456`)

Rôle	Adresse électronique	Remarque
Administrateur	admin@arp.edu	Compte fixe, seul non créé par l'application
Professeur	sami.benali@arp.edu	Responsable de DEVWEB et BDD (INFO-2A)
Professeur	nadia.trabelsi@arp.edu	Responsable de COMPTA (GEST-2A)
Étudiant	ahmed.j@arp.edu	Classe INFO-2A

Scénario de démonstration conseillé (10 minutes)

1. Admin : créer un étudiant par son courriel, montrer que son compte apparaît « à définir ».
2. Admin : ouvrir Emails, cliquer le lien d'activation, définir le mot de passe.
3. Étudiant : se connecter, consulter l'emploi du temps de la semaine, le télécharger en PDF.
4. Prof : faire l'appel sur une séance, ajouter deux notes CC/TP, créer un compte rendu avec date limite.
5. Étudiant : déposer son compte rendu (avant la limite).
6. Prof : clôturer la matière en attribuant les notes d'examen.
7. Admin : valider la matière, puis montrer que la moyenne publiée apparaît à l'étudiant.
8. Étudiant : créer une réclamation ; admin : la passer « en cours » puis « traitée ».
9. Admin : publier une notification, visible immédiatement sur les autres sessions.
10. Coupure de réseau : couper le serveur, montrer que l'emploi du temps consulté s'affiche toujours et que la page hors ligne présente la grille de la semaine.

12. Tests réalisés et résultats

Chaque ligne correspond à une exécution réelle sur la version produite, en HTTP local et en HTTPS sur l'adresse du réseau.

Vérification	Méthode	Résultat
Compilation et typage	`next build`, 29 routes générées	**Conforme**
Invitation complète : création, courriel, activation, connexion	Chemin réel avec extraction du jeton depuis la boîte d'envoi	**Conforme**
Jeton réutilisable après usage	Seconde soumission du même jeton	**Refusé**, message d'invalidité
Redirections de formulaire correctes après rechargement	Codes 303 sur les 13 points d'écriture	**Conforme**
Redirections depuis le téléphone (origine réseau, pas localhost)	Comparaison de l'en-tête `Location` entre appel par IP et appel par localhost	**Conforme** après introduction de `clientOrigin`
Planification hebdomadaire : enregistrement, suppression d'un créneau, copie	Écriture puis relecture de la grille	**Conforme**
Appel d'une séance (trois statuts)	Enregistrement puis comptage des occurrences affichées	**Conforme**
Correction d'un dépôt après expiration	Attribution d'une note et d'un commentaire	**Conforme**
Clôture d'une matière puis validation	Saisie de trois notes d'examen, publication, calcul de la moyenne pondérée	**Conforme** (moyenne de matière 14,00/20 pour un étudiant)
Confidentialité des notes avant validation	Consultation étudiant sur une matière « terminée »	**Conforme** — zone « en attente de publication »
Refus d'écriture hors rôle	Un étudiant tentant de créer une notification, un compte professeur, ou une salle	**403** sur les cas testés
Garde des pages et API sans session	Requête anonyme sur `/notes`	**Redirection** vers la connexion
Téléchargement du PDF de l'emploi du temps	GET authentifié, contrôle de l'en-tête `Content-Disposition`, ouverture du fichier dans Chrome	**Conforme** — page A4 paysage, accents corrects, grille complète
PDF demandé par un étudiant pour une autre classe	Paramètre `classe_id` falsifié	**Conforme** — le fichier renvoyé est celui de sa propre classe
Accès direct aux pages réservées à l'admin	Sessions professeur et étudiant sur `/users`, `/classes`, `/emails`	**Redirection** vers le tableau de bord avec motif
Soumission fabriquée sur l'emploi du temps	Matière d'une autre classe, puis salle inexistante	**Refusés** avec message d'erreur
Note saisie pour un étudiant d'une autre classe	Formulaire professeur détourné	**Refusé**
Clôture par l'admin et écriture après validation	Admin clôturant une matière attribuée, suppression d'une note d'une matière validée	**Refusés**
Installabilité	Manifest lu par le navigateur, service worker actif, cache peuplé	**Conforme** sur les deux origines
Mode hors ligne	Serveur arrêté, puis navigation sur une page vue et une page jamais vue	**Conforme** — page servie depuis le cache ; page hors ligne avec calendrier de la semaine
Confiance TLS sur l'adresse réseau	Navigateur sur `https://192.168.100.10:3443`	**Conforme** — contexte sécurisé, sans avertissement
Adaptation mobile	Fenêtre à 638 px de large	**Conforme** — barre basse visible, barre latérale masquée

Défauts rencontrés pendant le développement et leur correction

Défaut	Origine	Correction

Erreur 500 après chaque connexion et chaque formulaire	Redirection 307 rejouant le POST sur une page serveur	Redirection 303 systématique dans les 13 routes
Page Emploi du temps cassée en production	Gestionnaire d'événement passé dans un composant serveur	Bouton d'impression isolé dans un composant client
« Télécharger en PDF » ouvrait en réalité la fenêtre d'impression	Aucune bibliothèque de génération de PDF prévue ; l'export dépendait du poste client	Vrai fichier PDF produit par le serveur (`lib/pdf.ts` et la route `/api/emplois/pdf`), sans dépendance ajoutée
Le téléphone renvoyé sur une adresse injoignable	Origine issue de Next et non de la demande réelle	`lib/origin.ts` utilisé par toutes les redirections
Échec de type sur les horaires de créneaux	Objet constant à clés littérales indexé par un nombre	Typage explicite de la table d'horaires
Structure XML du canevas cassée	Balises non équilibrées après ajout de la barre mobile	Reprise de l'imbrication des deux modes de navigation
Erreurs de compilation sur les types implicites et les exports indisponibles en 13.5	API plus récente supposée disponible	Annotations explicites, bandeau de mise en page posé à la main

13. Limites connues et évolutions possibles

13.1 Limites assumées

- **Courriels simulés.** Le module consigne les messages dans la table `emails` avec leur lien : aucun serveur SMTP n'est contacté, ce qui permet une démonstration hors ligne et sans boîte réelle. Le passage à l'envoi véritable consiste à remplacer `lib/mail.ts` par un transporteur SMTP (nodemailer) en gardant la même signature.
- **Version de Next contrainte.** Le maintien en Node 18.13 fixe Next en 13.5 : pas d'actions serveur, dépendances figées.
- **Base mono-poste.** SQLite convient à un serveur unique. Le cache de connexion est mémoïsé par processus : une écriture depuis un second serveur ne sera pas vue immédiatement par le premier.
- **Installabilité Android liée au certificat.** L'autorité locale doit être installée sur chaque téléphone ; sans elle, HTTP ne permet ni installation ni hors ligne.
- **Hors ligne en lecture seule** (volontaire, cf. section 10.4).
- **Aucune file d'attente hors ligne, aucun test automatisé** : la qualité est établie par les vérifications manuelles et fonctionnelles listées en section 12.

13.2 Évolutions naturelles

1. Envoi réel des courriels avec modèle HTML et expiration temporelle des jetons.
2. Déploiement hébergé : migration de SQLite vers une base client-serveur, puis installation PWA sans certificat local.
3. Règle d'examen : signalement automatique au-delà d'un nombre d'absences ou de retards.
4. Calendrier des examens et attribution des salles de composition.
5. Bulletin imprimable par étudiant (mêmes styles d'impression que l'emploi du temps).
6. Hors ligne avec mise en file d'attente des pointages, rejouée à la reconnexion.
7. Tests automatisés (unités sur les routes et scénarios de bout en bout).

14. Annexe — arborescence

Chemin	Contenu
`app/`	15 pages (composants serveur) + `app/api/` 13 points d'écriture + mise en page racine et styles globaux
`components/`	Sidebar, PwaRegister, PrintButton
`lib/`	db.ts, auth.ts, mail.ts, origin.ts
`middleware.ts`	Garde des pages et API ; chemins publics : `/login`, `/reset`, `/offline`, `/api/auth/login`, `/api/auth/reset`
`public/`	`manifest.webmanifest`, `sw.js`, `icons/icon.png`
`data/`	`arp.db` (+ fichiers WAL), créé automatiquement, hors suivi de versions
`certs/`	Certificat mkcert, clé privée, autorité publique — hors suivi de versions
`scripts/`	`get-mkcert.ps1` (téléchargement), `make-cert.ps1` (régénération pour l'IP courante)
`server-https.js`	Exposeur HTTPS de la version produite, port 3443
`README.md` / `README-ANDROID.md`	Démarrage général ; procédure détaillée téléphone

Document généré le 19 octobre 2026 pour la version 2.0 d'ARP Éducation — Next.js 13.5.11, SQLite (better-sqlite3 9.6.0), TypeScript 5.9.3, Node 18.13.